Implementation Documentation of Quadruped Gait Controllers

Contents

1 MuJoCo Trot Controller **2**

2 MuJoCo Crawl Controller **8**

3 Additional Gait Controllers **12**

Chapter 1

MuJoCo Trot Controller

1.1 Purpose of the Script

This script implements a MuJoCo-based trot controller for the Unitree Go2 quadruped. The controller is structured as a velocity-tracking loop that maps a commanded body velocity

$$u = \begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix}$$

to joint-space commands using phase scheduling, stance-foot anchoring, swing-foot replanning, and analytic inverse kinematics.

The code is organized as a self-contained simulation script with the following responsibilities:

- load the Go2 MuJoCo model,
- initialize the robot in a standing configuration,
- assign gait phases to the four legs,
- compute stance and swing foot targets,
- solve leg inverse kinematics,
- send joint position commands,
- log motion metrics and plot debugging traces.

1.2 Model Loading and Simulation Setup

The script first loads the Go2 XML model:

```
MODEL_PATH = r"unitree_go2\go2_description.xml"

model = mujoco.MjModel.from_xml_path(MODEL_PATH)
data = mujoco.MjData(model)
model.opt.gravity[:] = np.array([0.0, 0.0, -9.81])
SIM_DT = model.opt.timestep
```

This creates the MuJoCo model and simulation data structures. Gravity is explicitly set to Earth gravity, and the simulation timestep is read from the model. The timestep is used later for loop timing, gait progression, smoothing, and debug plotting.

1.3 Robot Geometry and Kinematic Parameters

The leg geometry is defined through three link parameters:

$$l_{hip} = \texttt{HIP_OFFSET}, \qquad l_{thigh} = \texttt{THIGH}, \qquad l_{calf} = \texttt{CALF}.$$

The body geometry is expressed through body length and width:

$$L = \texttt{BODY_L}, \qquad W = \texttt{BODY_W}.$$

These values are used to define:

- nominal foot positions,
- hip locations in the body frame,
- inverse kinematics geometry,
- stance and swing reference locations.

1.4 Nominal Foot and Hip Frames

The script defines nominal foot locations for each leg in the body frame:

```
NOMINAL_FEET = {
    "FR": np.array([ BODY_L/2, -(BODY_W/2 + HIP_OFFSET), -BODY_HEIGHT]),
    "FL": np.array([ BODY_L/2,  (BODY_W/2 + HIP_OFFSET), -BODY_HEIGHT]),
    "RR": np.array([-BODY_L/2, -(BODY_W/2 + HIP_OFFSET), -BODY_HEIGHT]),
    "RL": np.array([-BODY_L/2,  (BODY_W/2 + HIP_OFFSET), -BODY_HEIGHT]),
}
```

These positions represent a nominal standing pose with each foot placed below its corresponding hip. The hips themselves are defined as body-frame offsets:

```
HIP_BODY = {
    "FR": np.array([ BODY_L/2, -BODY_W/2]),
    "FL": np.array([ BODY_L/2,  BODY_W/2]),
    "RR": np.array([-BODY_L/2, -BODY_W/2]),
    "RL": np.array([-BODY_L/2,  BODY_W/2]),
}
```

The helper function `nominal_foot_hip(leg)` converts the body-frame nominal foot position into a hip-relative foot target.

1.5 Joint and Actuator Mapping

The controller needs to map semantic leg names to MuJoCo joint indices and actuators. This is handled through the `joint_order` list and the `_name_to_id()` utility.

The joint ordering is:

- FR hip, thigh, calf,
- FL hip, thigh, calf,
- RR hip, thigh, calf,
- RL hip, thigh, calf.

For each joint, the script finds:

- the corresponding joint index,
- the position in `qpos`,
- the matching actuator index.

This mapping is critical because the controller ultimately writes joint targets into `data.ctrl`.

1.6 Standing Initialization

Before locomotion begins, the robot is initialized in a standing posture. The script computes standing joint angles by solving IK for the nominal foot positions:

```
standing_q = []
for leg in LEGS:
    x, y, z = nominal_foot_hip(leg)
    standing_q += list(leg_ik(x, y, z, IS_LEFT[leg]))
```

The base pose is initialized at a standing height and the quaternion is set to identity. The stance joint positions are written into `qpos`, and the same values are also loaded into `ctrl` so the servo error is initially close to zero.

This avoids a free-fall transient at the start of the simulation.

1.7 Inverse Kinematics Function

The leg inverse kinematics are implemented in closed form:

```
def leg_ik(x, y, z, is_left=False):
    l1 = HIP_OFFSET if is_left else -HIP_OFFSET
    yz = y * y + z * z
    if yz <= HIP_OFFSET ** 2:
        return 0.0, 0.0, 0.0

    theta1 = math.atan2(y, -z) - math.atan2(l1, math.sqrt(yz - HIP_OFFSET ** 2))
    d = x * x + yz - HIP_OFFSET ** 2
    D = (d - THIGH ** 2 - CALF ** 2) / (2 * THIGH * CALF)
    D = max(-1.0, min(1.0, D))
    theta3 = -math.acos(D)

    a = math.atan2(-x, math.sqrt(yz - HIP_OFFSET ** 2))
    b = math.atan2(CALF * math.sin(abs(theta3)), THIGH + CALF * math.cos(theta3))
    theta2 = -(a - b)
    return theta1, theta2, theta3
```

The first angle corresponds to the abduction/adduction joint, the second to the thigh pitch, and the third to the knee pitch. The clamp on `D` protects the inverse cosine from numerical overflow when the target foot lies close to the edge of the reachable workspace.

1.8 Gait Timing and Phase Logic

The gait is parameterized by:

$$T = \texttt{GAIT_PERIOD}, \qquad d = \texttt{STANCE_RATIO}, \qquad T_{stance} = Td.$$

The global gait phase is computed from elapsed time:

$$\phi_{global}(t) = \frac{t \bmod T}{T}.$$

Each leg uses a phase offset:

$$\phi_i = (\phi_{global} + \phi_{offset,i}) \bmod 1.$$

A leg is considered in stance if $\phi_i < d$, otherwise it is in swing.
For the trot controller, the diagonal pairing is encoded with:

$$\text{FR} \leftrightarrow \text{RL}, \qquad \text{FL} \leftrightarrow \text{RR}.$$

This produces a classic trot pattern where diagonal pairs alternate stance and swing.

1.9 Stance Foot Handling

During stance, the foot is assumed fixed on the ground. The body frame moves relative to that contact point, so the code computes the foot position in the hip frame using:

$$^{H_i}\mathbf{p}_F = R_z(-\psi)\left(\mathbf{p}_F^{world} - \mathbf{p}_{body}^{world}\right) - \mathbf{p}_{hip}^{body}.$$

The helper `stance_foot_in_hip_frame()` implements this operation. This is the core idea that keeps stance legs supporting the body while the robot advances.

1.10 Swing Foot Trajectory

When a leg enters swing, the foot is moved from its previous lift-off position to a new touchdown target. The script uses a smooth interpolation in the horizontal plane and a lifted trajectory in the vertical direction.

The helper:

```
def swing_foot_in_hip_frame(leg, s, start_hip, target_hip):
    s_smooth = s * s * (3.0 - 2.0 * s)
    xy = (1.0 - s_smooth) * start_hip[:2] + s_smooth * target_hip[:2]
    z0 = start_hip[2]
    zf = target_hip[2]
    z_lin = (1.0 - s_smooth) * z0 + s_smooth * zf
    z = z_lin + STEP_HEIGHT * (4.0 * s * (1.0 - s))
    return np.array([xy[0], xy[1], z])
```

This trajectory ensures:

- smooth liftoff,
- smooth touchdown,
- sufficient clearance above the terrain.

1.11 Touchdown Target Computation

The touchdown target is computed using a Raibert-style heuristic:

```
def compute_touchdown_body(leg):
    dx = (vx * T_STANCE) / 2.0
    dy = (vy * T_STANCE) / 2.0
    dtheta = (omega * T_STANCE) / 2.0
    p_nom = NOMINAL_FEET[leg]
    td_xy = Rz(dtheta) @ p_nom[:2] + np.array([dx, dy])
    return np.array([td_xy[0], td_xy[1], p_nom[2]])
```

The idea is that the stance phase lasts T_{stance}, so the next foothold should compensate for the expected body displacement over that time. The factor of $1/2$ is an implementation choice that moderates the stride length and helps prevent over-aggressive step placement.

1.12 Body Pose Handling

The script maintains an internal `body_pose` variable for kinematic planning:

```
body_pose[0] -= vx * SIM_DT
body_pose[1] += vy * SIM_DT
body_pose[2] = wrap_to_pi(body_pose[2] + omega * SIM_DT)
```

This pose is used for planning the relative geometry of feet and hips. It is not a full dynamic state estimator; rather, it is a simplified body-frame progression model for gait generation.

1.13 Control Loop Structure

The main loop performs the following steps:

1. hold a standing pose for a short settling duration,
2. compute the global gait phase,
3. determine stance or swing for each leg,
4. update stance foot positions and swing targets,
5. solve IK for each leg,
6. write joint commands,
7. step the MuJoCo simulation,
8. log debug metrics.

The structure is modular, and each leg is handled independently within the same gait phase logic.

1.14 Debug Plotting and Metrics

A background thread continuously plots:

- commanded and estimated forward velocity,
- commanded and estimated lateral velocity,
- commanded and estimated yaw rate,

- Froude number estimates.

The script also records a metrics log that includes:

- time,
- commanded and estimated velocities,
- yaw, roll, and pitch,
- settling-time proxies,
- drift indicators.

This is useful for tuning gait parameters and comparing trajectory quality across runs.

1.15 Implementation Summary

The trot controller is a kinematic velocity-tracking loop with:

- diagonal-phase scheduling,
- stance-foot anchoring,
- swing foot replanning,
- closed-form leg IK,
- MuJoCo position control,
- runtime debugging and performance logging.

Chapter 2

MuJoCo Crawl Controller

2.1 Purpose of the Script

This script implements a static crawl gait for the Go2 robot in MuJoCo. Compared with the trot controller, the crawl gait prioritizes stability over speed. The controller is designed so that the robot keeps three feet on the ground for most of the cycle while one leg swings at a time.

The main components are:

- phase-based crawl scheduling,
- lateral body-shift compensation,
- stance-foot anchoring,
- swing-foot trajectory generation,
- touchdown replanning,
- analytic IK,
- real-time debug visualization and metrics logging.

2.2 Simulation Initialization

The model loading and simulation initialization are identical in structure to the trot script:

```
MODEL_PATH = r"unitree_go2\go2_description.xml"

model = mujoco.MjModel.from_xml_path(MODEL_PATH)
data = mujoco.MjData(model)
model.opt.gravity[:] = np.array([0.0, 0.0, -9.81])
SIM_DT = model.opt.timestep
```

The robot is initialized in a standing configuration using the same general logic as in the trot controller.

2.3 Geometry and Gait Parameters

The crawl controller uses the same Go2 geometry:

$$l_{hip} = \texttt{HIP_OFFSET}, \qquad l_{thigh} = \texttt{THIGH}, \qquad l_{calf} = \texttt{CALF}.$$

The gait parameters are chosen to emphasize static stability:

$$T = \texttt{GAIT_PERIOD}, \qquad d = \texttt{STANCE_RATIO}, \qquad T_{stance} = Td.$$

A high duty factor is used so that the robot remains in contact with the ground for most of the cycle.

2.4 Crawl Phase Schedule

The crawl gait follows a standard four-step sequence:

$$RR \rightarrow FR \rightarrow RL \rightarrow FL.$$

The phase offsets are:

$$\phi_{offset,FL} = 0.00, \quad \phi_{offset,RL} = 0.25, \quad \phi_{offset,FR} = 0.50, \quad \phi_{offset,RR} = 0.75.$$

Each leg's local phase is computed in the same way as in the trot script:

$$\phi_i = (\phi_{global} + \phi_{offset,i}) \bmod 1.$$

This produces a single-swing-leg crawl in which only one leg is typically in swing at a time.

2.5 Lateral Stability Shift

A major difference from the trot script is the explicit lateral body shift. The controller moves the body slightly toward the stance side while a leg is swinging. This is done to keep the support polygon favorable and reduce the likelihood of tipping.

The code defines:

```
BODY_SHIFT_Y = 0.01
SHIFT_RAMP_TIME = 0.5 * (1.0 - STANCE_RATIO) * GAIT_PERIOD
```

The target shift is a simple two-mode function:

```
def lateral_shift_target(g_phase):
    if g_phase < 0.5:
        return BODY_SHIFT_Y
    else:
        return -BODY_SHIFT_Y
```

This shift is smoothed with a first-order filter:

$$y_{shift,k} = y_{shift,k-1} + \alpha(y_{target} - y_{shift,k-1}),$$

where α is derived from the timestep and ramp duration.

This shift is used only for inverse kinematics planning, not as a direct command to the physics engine.

2.6 Stance Foot Planning

As in the trot controller, the stance feet are treated as fixed world-frame contacts. When a leg is in stance, the foot position is converted into the hip frame of the current body pose, then sent into IK.

The important distinction is that in crawl gait the body pose used for planning includes the lateral shift:

$$\mathbf{p}_{body}^{plan} = \mathbf{p}_{body} + \begin{bmatrix} 0 \\ y_{shift} \\ 0 \end{bmatrix}.$$

This makes the stance legs compensate for the lateral lean while preserving the world-frame contact assumption.

2.7 Swing Foot Trajectory

The swing trajectory in the crawl controller uses an eased interpolation in the horizontal direction and a smooth height profile in the vertical direction.

The code uses:

```
def swing_foot_in_hip_frame(leg, s, start_hip, target_hip):
    """Refined swing trajectory utilizing an easing curve for smoother liftoff/
    touchdown."""
    s_smooth = s * s * (3.0 - 2.0 * s)
    xy = (1.0 - s_smooth) * start_hip[:2] + s_smooth * target_hip[:2]
    z0 = start_hip[2]
    zf = target_hip[2]
    z_lin = (1.0 - s_smooth) * z0 + s_smooth * zf
    z = z_lin + STEP_HEIGHT * (4.0 * s * (1.0 - s))
    return np.array([xy[0], xy[1], z])
```

This creates a smooth swing motion with a clearance hump in mid-swing.

2.8 Touchdown Placement Rule

The touchdown target is generated from the nominal foot placement using a body-motion heuristic:

```
def compute_touchdown_body(leg):
    dx = (vx * T_STANCE) / 0.8
    dy = (vy * T_STANCE) / 1.0
    dtheta = (omega * T_STANCE) / 1.0
    p_nom = NOMINAL_FEET[leg]
    td_xy = Rz(dtheta) @ p_nom[:2] + np.array([dx, dy])
    return np.array([td_xy[0], td_xy[1], p_nom[2]])
```

The purpose of this rule is to shift the foot forward enough so that, after the stance phase, the support pattern remains compatible with the commanded velocity. The scaling factor on `dx` is an implementation heuristic that tunes stride aggressiveness.

2.9 Crawl Loop Logic

The crawl loop contains the following stages:

1. maintain a standing pose for a short settling period,
2. compute the global gait phase,
3. apply the lateral body shift,
4. determine whether each leg is in stance or swing,
5. update swing lift-off and touchdown targets,
6. solve IK for all legs,
7. send joint commands,
8. step the simulation,
9. log tracking performance.

The overall structure is similar to the trot controller, but the crawl gait introduces more explicit stability handling.

2.10 Metrics and Debugging

The same debugging philosophy is used in the crawl controller:

- a live plot of velocity tracking,
- Froude number estimates,
- logged commanded and measured velocities,
- yaw and attitude traces.

These metrics help quantify how much of the commanded speed is actually realized in simulation and whether the crawl gait is dominated by forward progression, lateral oscillation, or yaw oscillation.

2.11 Implementation Summary

The crawl controller is a quasi-static kinematic gait generator with:

- a four-leg phase schedule,
- one-leg-at-a-time swing logic,
- lateral CoM shifting for stability,
- touchdown replanning from velocity commands,
- closed-form IK,
- MuJoCo position commands,
- runtime debug metrics.

Chapter 3

Additional Gait Controllers

3.1 Pace, Bound, and Pronk Controllers

The controller architecture used for the Trot and Crawl gaits easily generalizes to other symmetrical and asymmetrical gaits simply by altering the phase offsets (`PHASE_OFFSET`) and timing parameters.

3.1.1 Pacing Controller (`go2_mujoco_pace.py`)

Pacing synchronizes the legs on the same side of the body.

```
PHASE_OFFSET = {"FR": 0.0, "RR": 0.0, "FL": 0.5, "RL": 0.5}
```

3.1.2 Bounding Controller (`go2_mujoco_bound.py`)

Bounding synchronizes the front legs and the rear legs.

```
PHASE_OFFSET = {"FR": 0.0, "FL": 0.0, "RR": 0.5, "RL": 0.5}
```

3.1.3 Pronking Controller (`go2_mujoco_pronk.py`)

Pronking synchronizes all four legs to lift off and touch down simultaneously.

```
PHASE_OFFSET = {"FR": 0.0, "FL": 0.0, "RR": 0.0, "RL": 0.0}
```

3.2 Trot Variations: Walking Trot and Flying Trot

By keeping the trot phase offsets ($\phi_{FR} = \phi_{RL} = 0.0$, $\phi_{FL} = \phi_{RR} = 0.5$) but modifying the `GAIT_PERIOD` and `STANCE_RATIO` (duty factor), different trot behaviors are produced.

3.2.1 Walking Trot (`go2_mujoco_walking_trot.py`)

Used for slower, more stable locomotion.

```
GAIT_PERIOD = 0.5
STANCE_RATIO = 0.6  # > 0.5 ensures double support overlaps
```

3.2.2 Flying Trot (`go2_mujoco_flying_trot.py`)

Used for high-speed locomotion, introducing an aerial phase.

```
GAIT_PERIOD = 0.3
STANCE_RATIO = 0.4  # < 0.5 creates an aerial phase where all feet are airborne
```

3.3 Fine-Tuning the Raibert Touchdown Heuristic

In all controllers, the `compute_touchdown_body` function implements a Raibert-style heuristic:

```
def compute_touchdown_body(leg):
    T_swing = GAIT_PERIOD * (1.0 - STANCE_RATIO)
    T_horizon = T_STANCE / k + T_swing  # k is a tuned divisor
    dx = vx * T_horizon
    dy = vy * T_horizon
    dtheta = omega * T_horizon
    p_nom = NOMINAL_FEET[leg]
    td_xy = Rz(dtheta) @ p_nom[:2] + np.array([dx, dy])
    return np.array([td_xy[0], td_xy[1], p_nom[2]])
```

The divisor k (which is 2.0 in the standard Raibert heuristic) is fine-tuned per gait to improve velocity tracking accuracy (e.g., 1.2 for trot, 0.7 for static walk) to counteract drift and physical limits in open-loop IK execution.